UNIT-I

A Typical Progeamming task can be divided into 2 Phases.

Problem solving Phase:-

Problem solving Phase consists of sequence of steps that describes the solution of problem.

This sequence of steps is called an algorithm.

Implementation Phase:-

In Implementation Phase we will implement the program by using some programming language.

ex:- Construction of building.

Algorithm:-

Algorithm is a finite set of steps which are used to solve a particular problem or a task.

(or)

An algorithm is a sequence of computation steps that transform the input into the output

Properties of Algorithm:-

An algorithm must satisfy the following criteria.

(1) Input: zero or more quantities are externally supplied.

2) output :- At least one quantity is produced.

Definitness :-

each step of the algorithm must be clear and unambiguous.

Eg :- we cannot write divide 5/0 i.e 5/0 is not definit.

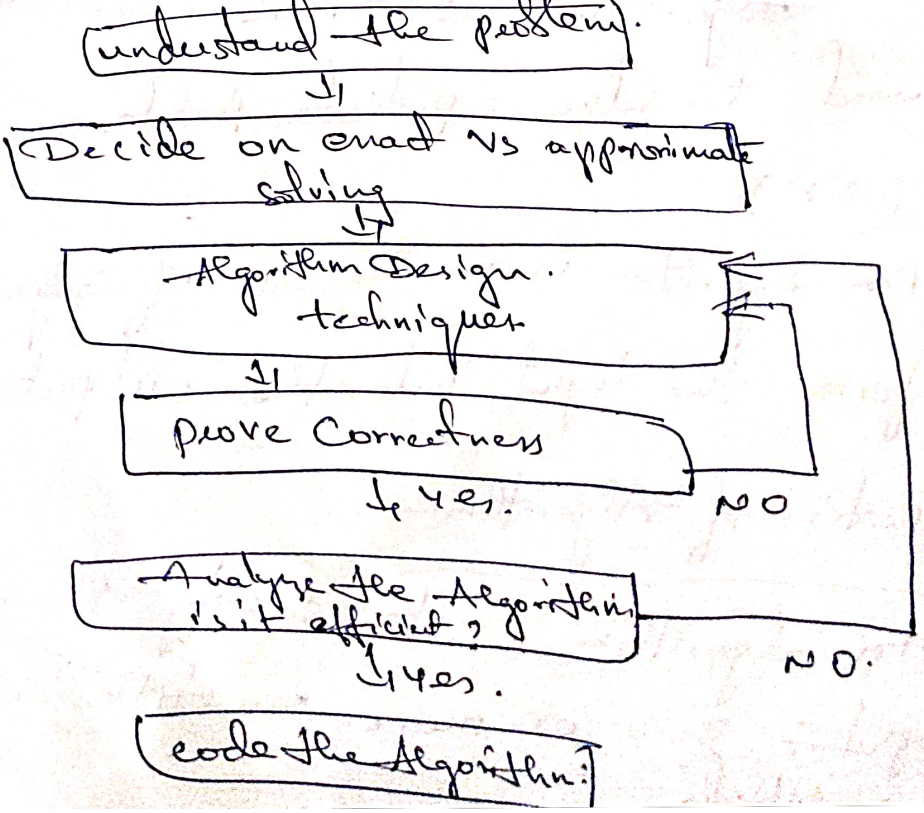
fitness (Termination)

Algorithm must terminate at a particular point, If it does not have a termination then it is not said to be an algorithm.

Effectiveness

Every step of algorithm must be effective i.e every operation should be done roughly by pen and paper.

Procedure for Design and Analysis of Algorithm.



① understand the Problem: Before designing an algorithm it is necessary to understand the problem completely. If we did any mistake it this step the entire algorithm becomes wrong.

② Exact Vs Approximate Solving.
solve the problem exactly if possible, otherwise use approximation methods.

③ Algorithm design Techniques:- In this we will use different design techniques like:
(i) Divide - and - Conquer.
(ii) Greedy method
(iii) Dynamic Progamming
(iv) Backtracking.
(v) Branch and Bound.
(vi) Brute - force etc.

④ Prove correctness:- Generally testing is used for Proving correctness

⑤ Analyse an Algorithm:- Analysing the algorithm means studying the algorithm behaviour, i.e, calculating the time complexity and space complexity.
for an algorithm the time complexity should be minimum.

⑥ coding an algorithm: After completion of all phases successfully, Then we will code an algorithm
coding should not depends on any program language.

Example:- Algorithm to find the average and grade of student marks.

Algorithm:-

step1: Input M_1, M_2, M_3, M_4
step2: GRADE $\leftarrow (M_1+M_2+M_3+M_4)/4$
step3: if(GRADE < 50) Then
Print "FAIL"
else
Print "PASS"
endif.

Algorithm specification:-

We can describe an algorithm in many ways.

(1) We can use a natural language like english lang.

(2) We can use graphic representation called flowcharts

(3) Pseudocode.

Pseudocode convention:-

Pseudocode is a combination of natural lang and high-level Programming concepts.

Pseudocode

(1) Comments begin with // and continue until the end of line.

(2) Blocks are indicated with matching braces: { and }. A compound statement can be represented as a block.

Ex:-

```
{
    stmt 1;
    stmt 2;
    :
}
```

3) All statements are separated by ';'

4) An identifier begins with a letter.

Ex: var1, var2, X, Y ...

5) Assignment of values to the variables is done using the Assignment operator.

:= or ←

6) There are two boolean values TRUE and FALSE

Logical operators - AND, OR, NOT

Relational operators - <, ≤, >, ≥, =, ≠

7) Elements of multidimensional arrays are allocated using [and]

Ex:- A[i, j]

8) Looping statements

(i) while loop

```
while (condition) do
{
    stmt 1;
```

```
    stmt 2;
      .
      .
    stmt n;
}
```

As long as <condition> is true. The statement get executed. when <condition> becomes false, the loop is ~~exited~~. exits

For loop:-

```
for variable := value 1 to value n stepstep do.
{
    statement 1;
    statement 2;
      .
      .
    statement n;
}
```

here value 1, value n and step are arithmatic expression. A variable of type integer, or real or numeric constant is simple form of an arithmetic expression. The clause step step is optional and taken as +1 if does not occur.

Repeat-until statement:-

```
repeat
    <stmt 1>
    & stmt 2>
      .
      .
    &stmt n>
until <condition n>.
```

The stmts are executed as long as condition is false. The value of condition is computed after executing the statement.

→ TO exit from the loop we can use break statement (instruction)

7.) A conditional statement has the following.

=) if (condition) then (statement)

if (condition) than (statement 1) else (statement 2)

=)

```
case
{
  :(condition 1) : (statement 1)
  :(condition2) : (statement 2)
        .
        .
  : else  :(statemat n+1)
}
```

10). Input and output are done using the instructions read and write.

11) Algorithm. is a procedure consisting of heading and body. The heading has the form.

Algorithm Name (parameter list)

Pm:-

1) write an algorithm which finds and returns the maximum of 'n' given numbers.

```
1. Algorithm Max(A,n)
2. // A is an array of size n
3. {
4.    Result := A[1];
5.    for i:=2 to n do
6.       if A[i]> Result then Result:= A[i];
7.    return Result;
8. }
```

Selection sort Algorithm.

In this sorting technique, we will find the smallest value in given array and store in respective place.

ex:-

	0	1	2	3	4	5
a	5	2	1	3	6	4

swap.

5 2 1 3 6 4. 1 - smallest.

1 2 5 3 6 4.

1 2 5 3 6 4

1 2 3 5 6 4
  i

1 2 3 4 6 5
    j

1 2 3 4 5 6.

```
1. Algorithm SelectionSort (a,n)
2. // sort the array a[1:n] into nondecreasing order.
3. {
4. for i=1 to n do.
5. {
6. min=i;
7. for j = i+1 to n do.
8. if a[j] < a[min] then min=j;
9. t = a[i]; a[i]=a[min]; a[min]=t;
10 }
11 }.
```

Algorithm for finding the GCD of 2 numbers
Simple Algorithm.

Input : 2 integers m, n
output : largest integer that divides both.

Algorithm

1. factorise m. Find primes $m_1, m_2 .. m_i$ s.t
$M = m_1 \times m_2 \times .. m_i$

2. factorize n;

$n = n_1 \times n_2 \times \cdots n_j$

3. Identify common factors, multiply and return result.

Euclid's Algorithm.

```
Euclid (m, n) {
    while m does not divide n.
        r = n mod m
        n = m
        m = r
    end.
    return m
}.
```

36 divides 48.

$r = 48 \bmod 38$

$= 12$

$n = 12$

$n = 36$

$m = 12$

---

12 divides 36

return 12

example :-

$m = 36, n = 48$

$m = 2 \times 2 \times 3 \times 3$ $\quad n = 2 \times 2 \times 2 \times 2 \times 3$ 2 divisions

common factors $= 2, 2, 3$

$G.C.D = 2 \times 2 \times 3 = 12$

Performance Analysis

To Analyse the efficiency of the algorithm we will consider 2 factors.

(1) Time complexity (Time)
(2) Space complexity (memory)

Space complexity :-

Space complexity of an algorithm indicates the amount of memory needed by the algorithm to run to completion.

(or)

The space complexity of a program is the no. of elementary objects that the program needs to store during its execution.

The space requirement $S(P)$ of an algorithm P is

$$S(P) = C + Sp$$

(1) C is the fixed part that is independent of the inputs and outputs
(2) Sp is the variable part. or instance characteristics

Example:

(1) Algorithm xyz(x, y, z)
{
return (x+y+y*z+(x+y+z))/(x+y)+40;
}

$\Rightarrow S(x,y,z) = 3+0 = 3$

One space each for x, y, z.

$\therefore$ Space complexity is $O(1)$

Ex 1-2

Algorithm ADD(x, n)

{

Total = 0.0;

for i = 1 to n do

Total = total + x[i];

Return Total;

}

Above algorithm is having instance characteristic the 'n'

$S(ADD) = (n+3)$

$\Rightarrow$ where n is for array [x]

3 → one each for n, i, and total

$\therefore$ Space complexity is $O(n)$

2) Time complexity :- It is the amount of time a program or algorithm takes for execution.

→ The time T(P) taken by a program P is the sum of the compile time and the run time.

$T(P) = \text{CompileTime}(P) + \text{RuntTime}(P)$

Time Complexity of a Program depends on the no. of additions, subtractions, multiplication, divisions, compares, load, stores and so on.

→ The above operations are called as Primitive operations.

$t_p(n) = C_a ADD(n) + C_s SUB(n) + C_m MUL(n) + C_d DIV(n) + \ldots$

where n denotes the Instance characteristics.

and C_a, C_s, C_m, C_d and so on. respectively denotes the time needed for an addition, subtraction, multiplication, division and so on.

Ex:-

Counting method:-

Algorithm for sum of n numbers.

```
Algorithm Sum(a,n)
{
    s = 0.0;
    for i := 1 to n do
        s = s + a[i];
    return s;
}
```

```
1  Algorithm sum (a, n)
2  {
3      s = 0.0;
4      count = count + 1; // count is a global variable initially it is
                          zero.
5.     for i = 1 to n do.
6.     {
7.         count = count + 1; // for 'for'
8.         s = s + a[i]; count := count + 1; // for assignment
9.     }
10.    count = count + 1; // for last time of 'for'
11.    count = count + 1; // for the return
12.    return s;
13. }
```

Note:-

```
for i = 1 to n do.
{
...
}
```

t = time.

c(n) = no. of time the code to be executed

$\Rightarrow T(p) = t * c(n)$

$\boxed{T(p) = n}$

```
=> for i = 1 to n do.
   for j = 1 to n do.
   {
   ...
   }.
```

$\Rightarrow \boxed{T(p) = n * n = n^2}$

Step table.

Statement	s/e	frequency	total steps.
1. Algorithm sum(a,n)	0	—	0
2. {	0	—	0
3. s = 0.0;	1	1	1
4. for i = 1 to n do.	1	n+1	n+1
5. s = s + a[i];	1	n	n
6. return s;	1	1	1
7. }.	0	—	0
Total			2n+3.

$$T(P) = O(n)$$

⇒ s/e indicates steps per execution.

⇒ frequency represents total number of times the statement is executed.

Ex ②

⇒ Algorithm for matrix addition.

statement	s/e	frequency	total steps.
1. Algorithm add(a,b,c,m,n)	0	—	0
2. {	0	—	0
3. for i = 1 to m do	1	m+1	m+1
4. for j = 1 to n do	1	m(n+1)	mn+m
5. c[i,j] = a[i,j] + b[i,j];	1	mn	mn
6. }.	0.	—	0.
Total			2mn+2m+1

⑧

Counting Statement method:-

```
1. Algorithm Add(a,b,c,m,n)
2. {
3. for i=1 to m do.
4. {
5.   count=count+1; //for 'for i'
6. for j=1 to n do.
7. {
8.   count=count+1; //for 'for j'
9.   c[i,j]=a[i,j]+b[i,j];
10   count=count+1; //for the assignment
11 }
12   count=count+1; //last time of 'for j'
13 }
14   count=count+1;
15. }
```

Example 3:-

write an algorithm and find time complexity for sum of n numbers using recursive function.

```
1. Algorithm Rsum(a,n)
2. {
3.   if(n<=0) then return 0.0;
4.   else return Rsum(a,n-1)+a[n];
5. }
```

Counting statement method.

```
1. Algorithm Rsum(a,n)
2. {
3.   count = count + 1; // for the 'if' conditional.
4.   if(n <= 0) then.
5.   {
6.     count = count + 1; // for the return.
7.     return 0.0;
8.   }
9.   else.
10.  {
11.    count = count + 1; // for the addition, function
                          // invocation and return.
12.
13.    return Rsum(a,n-1) + a[n];
14.  }
15. }
```

⇒ From the above algorithm we obtain a recursive formula for the step count.

$$t_{Rsum}(n) = \begin{cases} 2 & \text{if } n = 0 \\ 2 + t_{Rsum}(n-1) & \text{if } n > 0 \end{cases}$$

⇒ These recursive formulas are referred to as recurrence relations.

$$t_{Rsum(n)} = 2 + t_{Rsum}(n-1)$$
$$= 2+2+t_{Rsum}(n-2)$$
$$= 2(2) + t_{Rsum}(n-2)$$
$$\vdots$$
$$= n(2) + t_{Rsum}(0)$$
$$= 2n+2 \qquad n \geq 0.$$

$\Rightarrow$ so the step count for Rsum is $2n+2$

step table.

Statement	s/e	frequency n=0	frequency n>0	total steps n=0	total steps n>0
1. Algorithm Rsum(a,n)	0	—	—	0	0
2- {	0				
3. if (n ≤ 0) then.	1	1	1	1	1
4. return 0.0;	1	1	0	1	0
5. else return	0	—	—	—	—
6. Rsum(a,n-1)+a[n];	1+x	0	1	0	1+x
7. }	0	—	—	0	0
Total.				2	2+x

∴ where $x = t_{Rsum(n-1)}$

Best case, worst case and Average case Efficiencies.

1.) Best case :- It is the minimum no. of steps that can be executed for a given parameter.

2.) Worst case :- It is the maximum no of steps that can be executed for a given parameter.

3. Average case :- It is the average number of steps executed for a given parameter.

Example ① : Linear search.

A

2	4	5	6	8	9	10	11	13
1	2	3	4	5	6	7	8	9

Best case :- Searching an element 2.

Time complexity is $O(1)$

Average case :- Searching an element 8.

Time complexity is $O(n/2) = O(n)$

Worst case :- Searching and element 13,

Time complexity is $O(n)$

example 2 :- Binary search.

Best case → $T(B) = O(1)$

Average case → $T(B) = O(\log_2 n)$

Worst case → $T(B) = O(\log_2 n)$

⇒ Tower of Hanoi Problem

$\log n < n < n\log n < n^2 < n^3 < 2^n < n!$ *

n	log n	nlogn	n^2	n^3	2^n	n!
1	0	0	1	1	2	1
2	1	2	4	8	4	2
4	2	8	16	64	16	24
8	3	24	64	512	256	40320
16	4	64	256	4096	65536	20922789 888000
⋮						

⇒ The function log n grows very slowly compared to any other function with respect to n. But 2^n and n! grows very fast.

⇒ when the algorithms are exponential in nature that is 2^n and n!, They can be solved practical with small values of n.

Definition: Asymptotic notation is a formula way notation to speak about functions and classify them.

They are five types of notation.

(i) Big 'OH' notation (O)
(ii) Omega notation (Ω)
(iii) Theta notation (Θ)
(iv) Little 'oh' notation (o)
(v) Little omega notation (ω)

Asymptotic Notation (O, Ω, Θ)

After designing the algorithm, we have to analyse the time taken on RAM model.

example :-

Ram Time $10n^3 + 5n^2 + 7$ "cubic in n"

$2n^3 + 3n + 79$ "cubic"

The above functions are categorized under same class i.e cubic.

⇒ The constant multiplies should be ignored.

The order of growth.

⇒ If algorithms are faster for smaller values of n and slower when n is larger then we can say that these algorithms are not good. This is what we call the order of growing n.

Common Time functions.

function	Name.
1	constant
$\log n$	Logarithmic
n	Linear
$n \log n$	$n \log n$.
n^2	quadratic
n^3	cubic
2^n	exponential
$n!$	factorial.

1.) Big "Oh" notation (O):-

The function $f(n) = O(g(n))$ if and only if there exists a positive constant c and n_0 such that

$f(n) \leq c \cdot g(n)$ for all n, where $n \geq n_0$

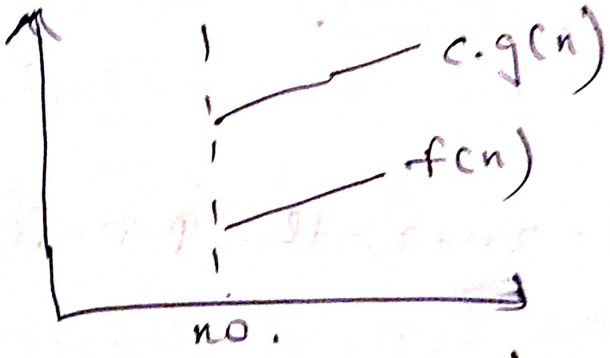
$O(g) = \{ f \mid f$ is non negative function s.t.

$\exists\ c, n_0$ s.t

$f(n) \leq c\, g(n)$ for $n \geq n_0 \}$

→ f, g are non-negative functions of non-negative arguments.

⇒ f is a class of similar functions.



$f(n) = O(g(n))$

Big "Oh" notation denotes the upper bound of the given function.

⇒ The upper bound of $f(n)$ indicates that the function $f(n)$ will be the worst case that it does not consume more than this computing time.

Note:-

If we know.

This is the.

~~taken by~~ the algorithm.

$$T(n) \leq 15n^3 + 7n^2 + 35 \leq 57n^3.$$

Then we conclude $\Rightarrow T(n) \in O(n^3)$

$$f(n) \leq c(g(n))$$

$\Rightarrow$ if $T(n) \geq 2n^3 + 37$

$$T(n) \in \Omega(n^3)$$

$\Rightarrow$ from the above we conclude that

$$T(n) \in \Theta(n^3)$$

Examples:-

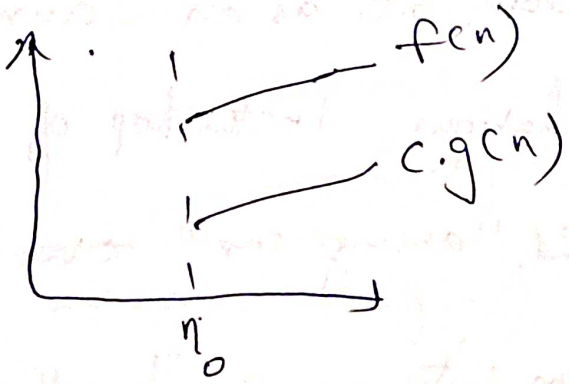
(1) Given $f(n) = 3n+2$, then P.T $f(n) = O(n)$

$\Rightarrow$.

2. Omega notation (Ω)

The function $f(n) = \Omega(g(n))$ iff there exist a positive constants c and n_0 such that

$f(n) \geq c \cdot g(n)$ for all n, $n \geq n_0$.



$f(n) = \Omega(g(n))$

Examples:-

(1) $f(n) = 3n+2$ then P.T $f(n) = \Omega(1)$

$3n+2 \geq 3n$, $n \geq 1$

when $c = 3$, $n_0 = 1$

(2) $100n+6 \geq 100n$, $n \geq 1 \Rightarrow \Omega(n)$

(3) $10n^2+4n+2 \geq n^2$, $n \geq 1 \Rightarrow \Omega(n^2)$

(4) $6 \times 2^n + n^2 \geq 2^n$, $n \geq 1 \Rightarrow \Omega(2^n)$

note:-

Big omega notation denotes the lower bound of the given function $f(n)$

→ The lower bound $f(n)$ indicates that the function $f(n)$ will be the best case.

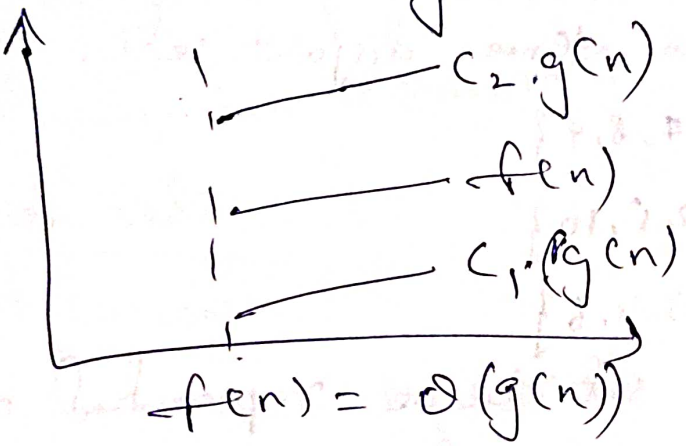
Randomized Algorithm:-

Is an Algorithm that employs a degree of randomness as part of its logic. The algorithm typically uses uniformly random bits as an auxilary input to guide its behaviour, in the hop of achieving good performance in the "average case" over all possible choices of random bits. formally, the algorithm's performance will be a random variable determined by the random bits, thus either the running time or, the o/p (or both) are random variables.

3.) Theta notation: (Θ)

The function $f(n) = \Theta(g(n))$ iff there exists a positive constants c_1, c_2 and n_0 such that

$$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \quad \forall n,\ n \geq n_0.$$



Examples:-

(1) $f(n) = 3n+2$ then P.T $f(n) = \Theta(n)$

$3n+2 \leq 4n$

$3n+2 \geq 3n \quad \forall n,\ n \geq 2$

$\Rightarrow 3n \leq 3n+2 \leq 4n \quad \forall n \geq 2$

where $c_1 = 3$

$c_2 = 4.$

$n_0 = 2.$

$\therefore$ here $f(n) = \Theta(n)$

SETS AND DISJOINT SET UNION.

Disjoint sets :-

The sets are the numbers 1, 2, 3 -- 7

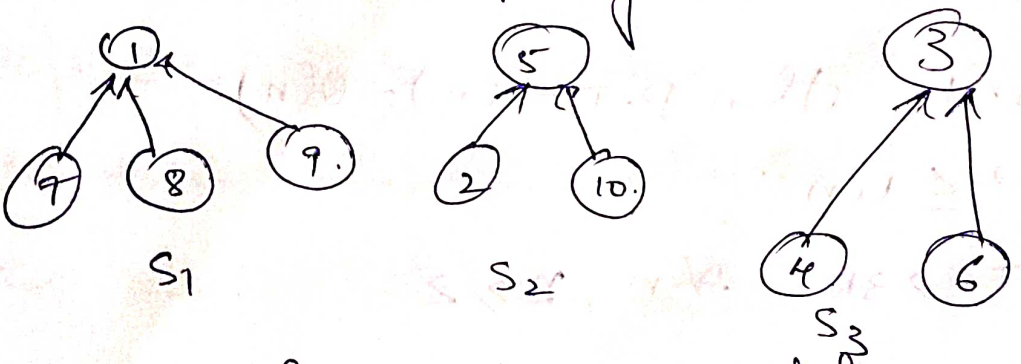
Consider a set $S = \{1, 2, 3, 4 . . 10\}$. These elements can be partitioned into three disjoint sets.

$S_1 = \{1, 7, 8, 9\}$

$S_2 = \{2, 5, 10\}$

$S_3 = \{3, 4, 6\}$

Each of the set can be represented as a tree, hence these trees corresponding to the sets are.



S_1 S_2 S_3

⇒ These sets can be represented by storing every element of the set in the same array.

⇒ The i^{th} element of this array represents the tree node that contains element i, This array element gives the Parent pointer of the corresponding tree node.

The following figure shows the array representation of the sets S_1, S_2 and S_3.

i	1	2	3	4	5	6	7	8	9	10.
p	-1	5	-1	3	-1	3	1	1	1	5

Array representation of sets S_1, S_2 and S_3.

Note :- Root nodes have a parent of -1

Operations on sets

(1) Disjoint set UNION :-

If S_i and S_j are two disjoint sets, Then the union $S_i \cup S_j$ = all elements x such that x is S_i or S_j

Thus $S_1 \cup S_2 = \{1, 7, 8, 9, 2, 5, 10\}$

(2) find(i):

print s The name of the set of which i is currently a member.

Ex :- $S_1 = \{1, 2\}$ $S_2 = \{4, 6\}$

FIND(4), This returns S_2, since $4 \in S_2$

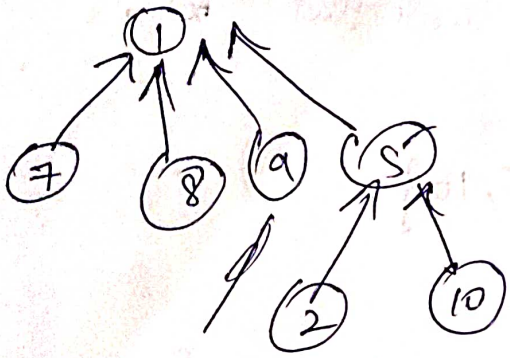
UNION and FIND algorithms :-

UNION operations :-

To obtain the union of S_1 and S_2. Since we have linked the nodes from children to parent.

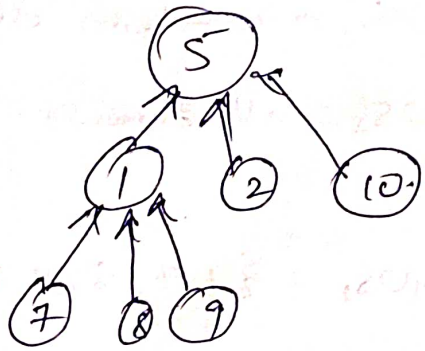
we simply make one of the trees as a subtree of the other.

$S_1 \cup S_2$ could then have one of the representation is follows.



$S_1 \cup S_2$

(or)



$S_1 \cup S_2$

→ The UNION(i,j), it means the element of set i and element of set j are combined of we want to represent UNION operation in the form of a tree.

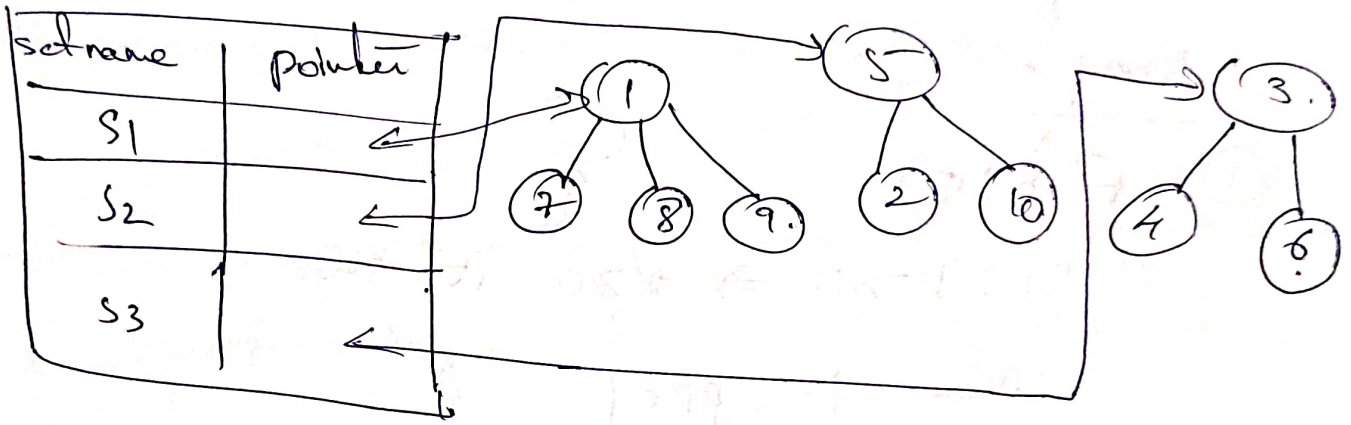
Example :-

UNION (1,3)

UNION (2,5)

The sets contain setname and then each has a pointer to the setname then to determine which set an element is currently in.

We follow parent links to the root of tree and use the pointer to the setname the data representation for S_1, S_2 and S_3



Algorithm UNION:-

1. Algorithm simpleunion (i, j)
2. {
3. P [i] := j;
4. }

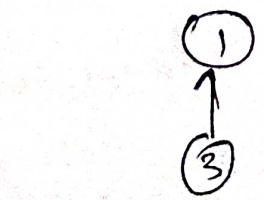
Example:-

i = 3 j = 1

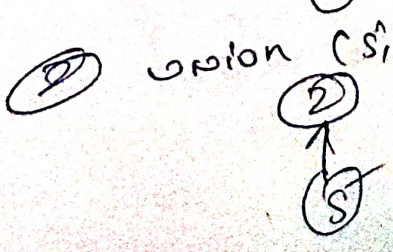
(1) UNION (3,1)

P[3] := 1

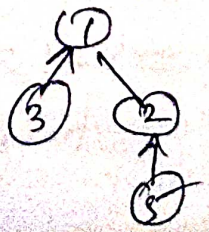
Parent of 3 is 1



(2) UNION (5,2)



(3) UNION (2,1)



Algorithm for Find (i):

```
1. Algorithm Simple Find(i)
2. {
3. while (p[i]>=0) do i:=p[i];
4. return i;
5. }
```

Example 1-

(1) Find (5)

$\Rightarrow p[5] \geq 0 \Rightarrow 2 \geq 0$ is true.

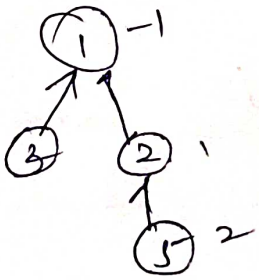
Then $i := p[5]$

$i = 2$

$p[2] \geq 0 \quad -1 \geq 0$ Not true.

So return 2.

(2) Find (5)



$p[5] \geq 0 \Rightarrow 2 \geq 0$, true.

$i = 2$

$p[2] \geq 0 \Rightarrow 1 \geq 0$, true.

$i = 1$

$p[1] \geq 0 \Rightarrow -1 \geq 0$, Not true.

return 1

The time taken by the UNION operation is

UNION $(x_1, x_2) \rightarrow O(1)$

UNION $(x_2, x_3) \rightarrow O(2)$

⋮

UNION $(x_{n-1}, x_n) \rightarrow O(n-1)$

UNION $(x_{n-1}, x_n) \rightarrow O(n-1)$

The union's cost is $1+2+\cdots+n-1 = \frac{n(n+1)}{2}$

$= O(n^2)$

⇒ We can improve the performance of our union and find algorithms by avoiding creation of degenerate trees.

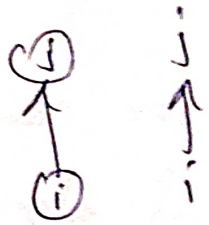
→ To accomplish this we make use of a weighting rule for union(i,j) and collapsing rule for find(i).

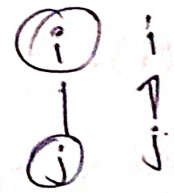
weighting rule for union(i,j)

If the number of nodes in the tree with root i is less than the number in the tree with root j then make jth parent of i, otherwise make ith parent of j.

Algorithm:-

```
Algorithm weighted union(i,j)
//union sets with roots i and j, i≠j
//using the weighted rule P[i] = -count[i]
// and P[j] = -count[j];
{
    temp := P[i] + P[j];
    if (P[i] > P[j]) then
    { // i has fewer nodes
        P[i] = j;
        P[j] = temp;
    }
    else
    { // j has fewer or equal nodes.
        P[j] = i;
        P[i] = temp;
    }
}
```





Example:-

[-1]	[-1]	[-1]	[-1]	[-1]	[-1]	[-1]	[-1]
①	②	③	④	⑤	⑥	⑦	⑧

Initial weight -1 trees.

Example :-

(1) union (1,2)

temp = -2

else.

P(2) = 1

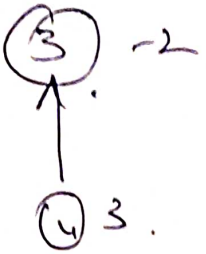
(2) union(3,4)

temp = -2

else,

P(4) = 3

P(3) = -2



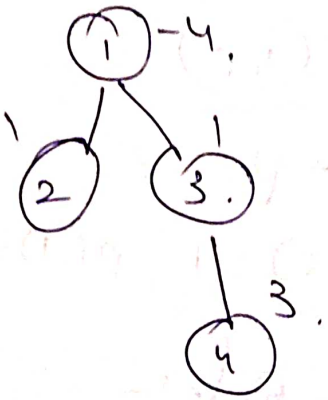
(3) union(1,3)

temp = -2 + -2 = -4.

else,

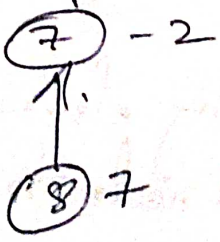
P(3) = 1

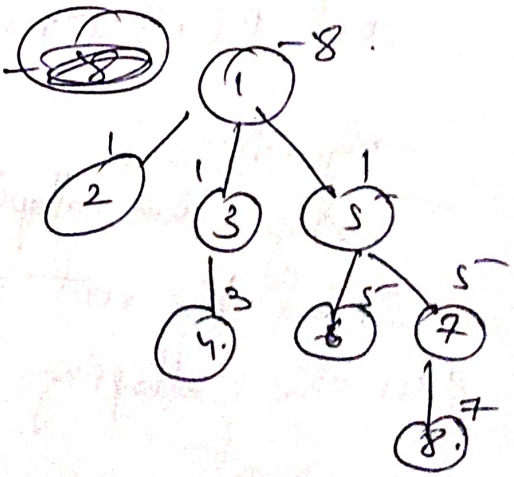
P(1) = -4.



(4) union(7,8)

temp = -2



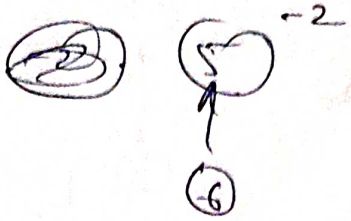


(5) union (5, 6)

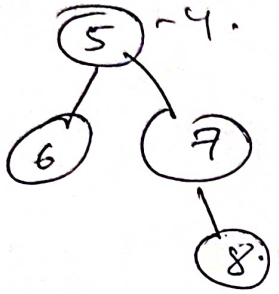
temp = -2

P(6) = 5

P(5) = -2.



(6) union (5, 7)



(7) union (1, 5)

temp = -8

P(5) = 1, P(1) = -8.

Collapsing Rule!-

If j is a node on the path from i to its root and P[i] ≠ root[i], then set P[j] to root[j]

Algorithm!-

Algorithm collapsingFind(i)

// find the root of the tree containing element i,

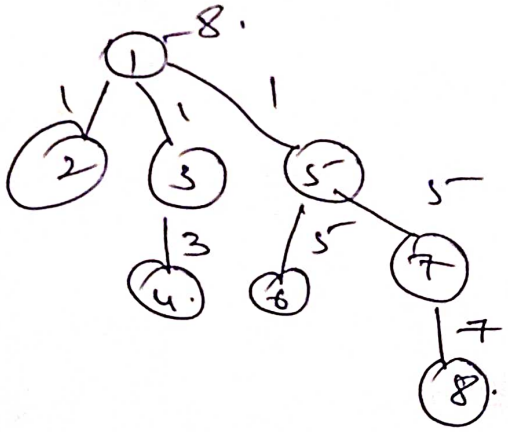
// use the collapsing rule to collapse all nodes from i to the root

{

r := i;

```
while (p[r] > 0) do.
{
   r := p[r]; //find the root
}
while (i ≠ r) //do collapse node from i to root r
{
    s := p[i];
    p[i] := r;
    i := s;
}
return r;
}
```

Example:-



find(8)

r = 8.

p[8] > 0

8 > 0, r = 7

at last r = 1

=> if r => 8 ≠ 1

s => p[8] = 7

p[8] = 1

$i = 7 \quad i \neq r \quad 7 \neq 1$

$\Rightarrow 7 \neq r.$

$s = p[7] = 5$

$s = 5$

$\boxed{p[7] = 1}$

$i \geq 5$

$\Rightarrow s = 1$

$p[5] = 1$

$i = 1$

$\Rightarrow i = 1, r = 1$

$1 = 1$, ~~false~~.

return 1.

DIVIDE-AND-CONQUER.

→ Given a function to complete on n inputs the divide-and-conquer strategy suggests splitting the inputs into k distinct subsets. $1 < k \leq n$, yielding k subproblems. These subproblems must be solved and then a method must be found to combine sub solutions into a solution of the whole.

→ If the subproblems are still relatively large, Then the divide-and-conquer strategy can possibly be reapplied.

→ The principle behind the divide-and-conquer algorithm design paradigm is that it is easier to solve several small instances of a problem than one large one.

→ The divide and conquer principle has three steps.

(1) Divide: In this step we will divide the problem into a no. of subproblems

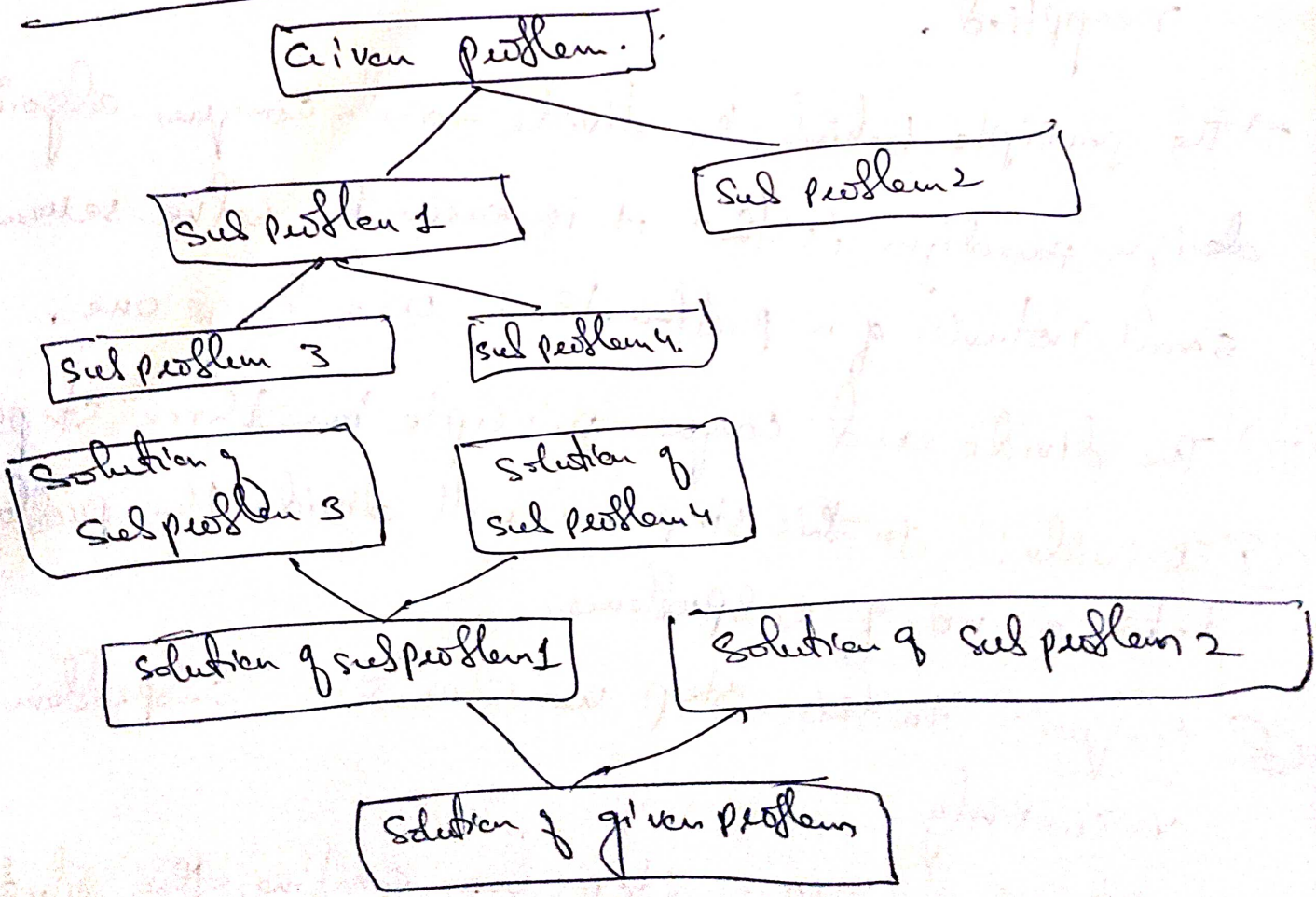
(2) Conquer:- In this step we solve the subproblem recursively

(3) Combine:- In this step we combine the solution of the subproblem to get the solution for the original problem.

Example :-

Asymptotic complexity for addition of 2 matrix.

Statement	s/e	frequency	total steps
Algorithm Add(a,b,c,m,n	0	—	$\Theta(0)$
{	0	—	$\Theta(0)$
for i:=1 to m do.	1	$\Theta(m)$	$\Theta(m)$
for i:=1 to n do.	1	$\Theta(mn)$	$\Theta(mn)$
c[i,j] = a[i,j]+b[i,j]	1	$\Theta(mn)$	$\Theta(mn)$
}	0	—	$\Theta(n$
Total			$\Theta(mn)$



Control abstraction for divide-and conquer.

```
1. Algorithm DAandC(P)
2. {
3.   if small(P) then return S(P);
4.   else
5.   {
6.     divide P into smaller instances P1, P2, ... Pk k≥1
7.     Apply DAandC to each of these subproblems;
8.     return combine(DAandC(P1), DAandC(P2)....
                      DAandC(Pk));
9.   }
10. }
```

DAandC Recurrence Relations:-

Let T(n) be the time complexity of algorithm then.

$$T(n) = \begin{cases} g(n) & \text{if } n \text{ is small.} \\ T(n_1) + T(n_2) + \cdots T(n_k) + f(n) & \text{otherwise.} \end{cases}$$

where,

g(n) → The time required to solve the problem. If n is small.

f(n) → The time required for dividing the problem 'P' into several subproblems.

The complexity of many divide-and-conquer algorithm is given by recurrence relation.

$$T(n) = \begin{cases} T(1) & \text{when } n=1 \\ aT(n/b)+f(n) & \text{when } n>1 \end{cases}$$

→ where a, b are known constants.

n is a power of b i.e $n = b^k$

⟹ a → number of subproblems.

n/b → subproblem size.

Example:- Consider the given problem and calculate the time complexity by using recurrence relation.

$a = 2$

$b = 2$

$T(1) = 2$

$f(n) = n$

Sol:-

$$T(n) = 2T(n/2)+n$$

$$= 2[2T(n/4)+n/2]+n$$

$$= 4T(n/4)+n+n$$

$$= 4T(n/4)+2n$$

$$= 4[2T(n/8)+n/4]+2n$$

$$= 8T(n/8)+n+2n$$

$$= 8T(n/8)+3n$$

$= 2^k T(n/2^k) + kn$

$= nT(n/n) + \log_2 n \cdot n$

$n = 2^k$
$\log n = \log_2 2^k$
$\log n = k.$

$= nT(1) + n\log_2 n$

$= 2n + n\log_2 n$

$\therefore O(n \log n)$

Example:-

$T(n) = 2T(n/2) + 2$
$T(1) = 0$
$T(2) = 1$

$1 + x + x^2 + \cdots x^{n-1} = \frac{a(r^n - 1)}{r - 1}$

Sol:-

$T(n) = 2T(n/2) + 2$

$= 2[2T(n/4) + 2] + 2$

$= 2^2 T(n/4) + 2^2 + 2$

$= 2^2(2T(n/8) + 2) + 2^2 + 2$

$= 2^3 T(n/2^3) + 2^3 + 2^2 + 2$

$\vdots$

$= 2^{k+1} T(n/2^{k-1}) + 2^{k-1} + \cdots + 2^2 + 2 + 1 - 1$

$= \frac{2^k}{2} \cdot T\left(\frac{n \cdot 2}{2^k}\right) + \frac{2^k - 1}{2 - 1} - 1$

$= \frac{n}{2} \cdot T(2) + n - 2$

$= \frac{n}{2}(1) + n - 2$

$= \frac{3n}{2} - 2$

$\therefore T(n) = O(n)$

or.

$\Rightarrow 2^k T(n/2^k) + 2^k + 2^{k-1} + \cdots 2^2 + 2$

$\Rightarrow n T(n/n) + 2^k + \frac{2^k - 1}{2-1} - 1$

$\Rightarrow n T(1) + n + \frac{n-1}{2-1} - 1$

$\Rightarrow n \times 0 + n + n - 1 - 1$

$= 2n - 2$

$\therefore T(n) = O(n)$.

Application of divide and conquer method.

* Binary search
* merge sort
* Quick sort
* fibonacci series
* matrix multiplication.
* factorial
* Strassen's algorithm etc.

Merge sort :-

Merge sort is an example of divide-and-conquer method. It works by dividing the given array into two sub arrays of equal size.

The sub arrays are sorted independently using recursion.

The sorted sub arrays are then merged to get the solution for the original array.

Algorithm mergesort (a, first, last)
// a [first : last] is global array to be sorted
// Input parameters : a, first, last (a[first . . last])
// output parameters : array a[first . . last] sorted in
non-decreasing order.

{
if (first == last)
return a // if only one element, simply return
else // divide a into two nearly equal parts.
{
mid = (first + last) / 2; // sort each half.
mergesort (a, first, mid);
mergesort (a, mid+1, last);
merge (a, first, mid, last); // Merge the two sorted arrays.
}
} // End merge sort.

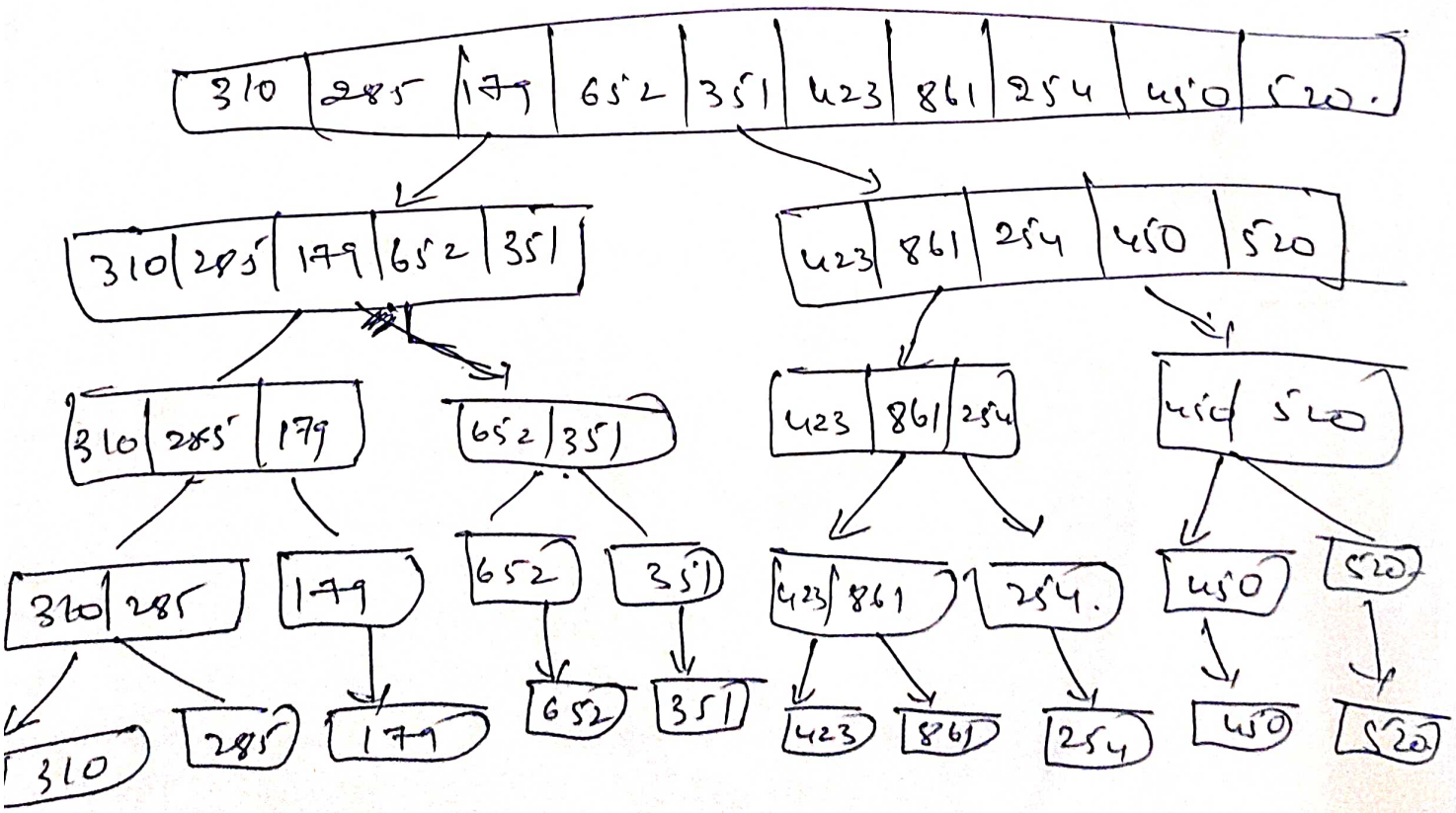
```
Algorithm Merge(a, first, mid, last)
// Merge two sorted subarrays into one sorted array
// Input parameters : a, first, mid, last
// Output parameter : sorted array a[first..last].
{
  p:=first // index in first subarray a[first:mid]
  q=mid+1 // index in second subarray a[mid+1:last].
  r=first // index in a local array b[].
  while((p<mid && (q<last))
  {
    if(a[p]<a[q]) // copy smaller value to b
    {
      b[r] = a[p];
      p = p+1;
    }
    else
    {
      b[r]=a[q]
      q = q+1;
    }
    r = r+1;
  }
  while(p<mid) // copy remainder, if any 1st sub
      array to b.
  {
    b[r] = a[p];
    p = p+1;
    r = r+1;
  }.
```

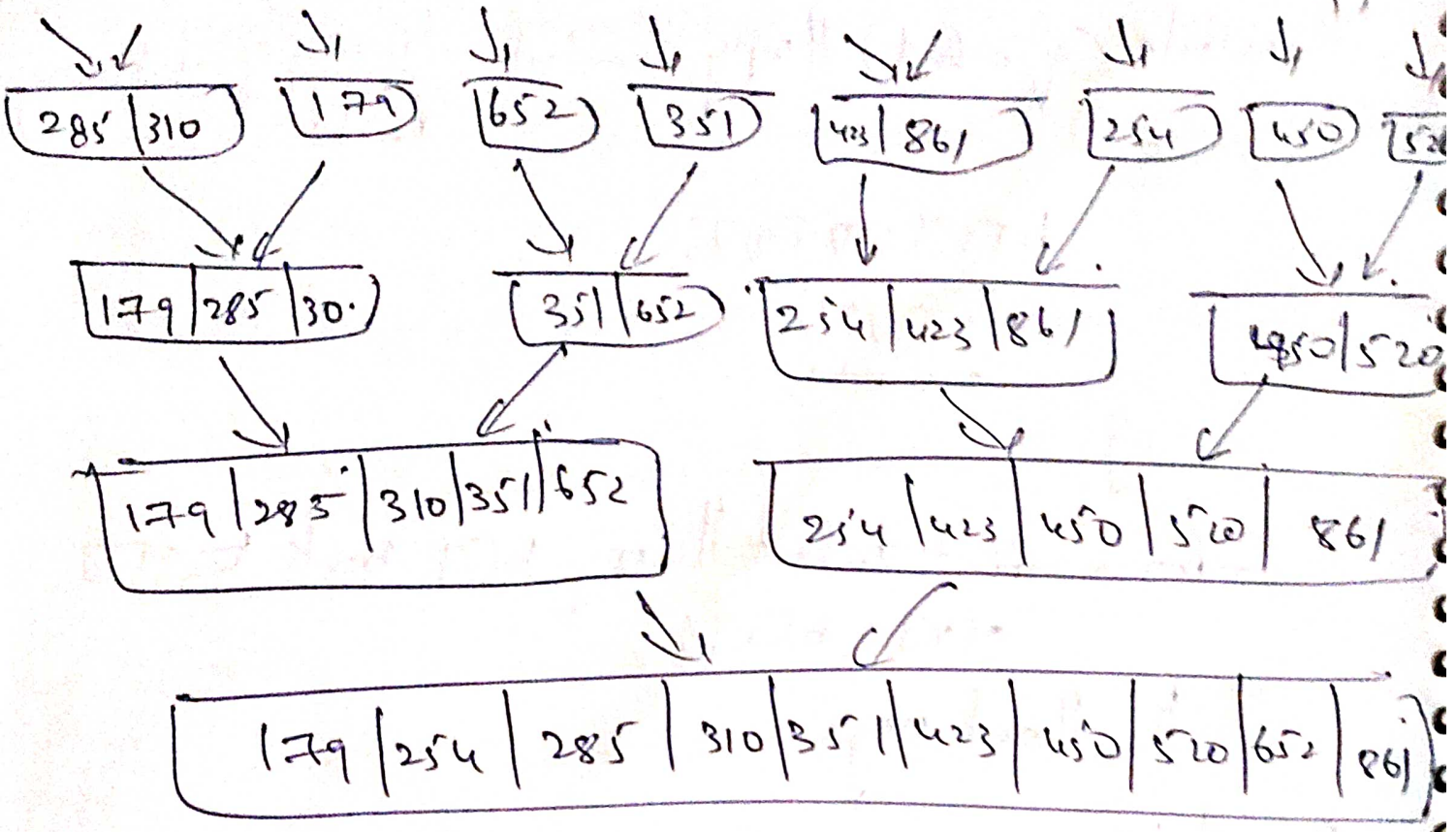
```
while (q < last) // copy remainder, if any, of subarray to b.
{
    b[r] = a[q];
    q = q+1;
    r = r+1;
}

for r = first to last // copy b[] back to a[]
    a[r] = b[r];
} // end merge.
```

Example:-

consider the array of ten elements

a[1:10] = (310, 285, 179, 652, 351, 423, 861, 254, 450, 520)




285 | 310
179
652
351
423 | 861
254
450
179 | 285 | 30
351 | 652
254 | 423 | 861
450 | 520
179 | 285 | 310 | 351 | 652
254 | 423 | 450 | 520 | 861
179 | 254 | 285 | 310 | 351 | 423 | 450 | 520 | 652 | 861

a[1:10]

Recurrence Relation of merge sort

$$T(n) = \begin{cases} a & n=1 \quad \text{'a' a constant} \\ 2T(n/2)+cn & n>1, \ c, \text{ a constant} \end{cases}$$

$$T(n) = 2T(n/2)+cn$$

$$= 2\left[2T(n/4)+c.\lfloor n/2 \rfloor\right]+cn$$

$$= 4T(n/4)+2.c.n/2+cn$$

$$= 4T(n/4)+2cn$$

$$= 4\left[2T(n/8)+c.n/4\right]+2cn$$

$$= 8T(n/8)+cn+2cn$$

$$= 8T(n/8)+3cn$$

$$\vdots$$

$$= 2^k T(n/2^k)+kcn$$

$$= nT(n/n)+\log_2 n \cdot cn \qquad \because n=2^k.$$

$$= nT(1)+\log_2 n \cdot cn \qquad \boxed{\log_2 n = k}$$

$$= an+cn\log n.$$

$$\boxed{T(n) = O(n\log n)}$$

Quicksort :-

→ Quick sort technique is based on the divide and conquer design technique.

→ In this technique at every step each element is placed in its proper position.

→ It performs very well on longer lists.

→ It works recursively, by first selecting a random "pivot value" from the list (array). Then it partitions the list into elements that are less than the pivot and greater than the pivot.

→ The problem of sorting a given list is reduced to the problem of sorting two sublists and process continuous until the list is sorted

→ This sorting technique is considered as a in-place since it uses no other array storage.

Divide and conquer procedure for Quick sort :-

Divide :- A[P--r] is divided into two halves A[P--q-1] and A[q+1---r] to be solved recursively. Here, A[q] is the pivot element as shown below

A[P]..A[q-1] A[q] A[q+1]..A[r].
↓↓ pivot

All are ≤ A[q] All are ≥ A[q].

~~Divide or~~

Conquer:- A[p..q-1] and A[q+1..r] are sorted recursively

Combine:- No need of this step, as all the leave sorted array in-place.

~~Algorithm~~:-

~~Algorithm~~ Quicksort (first, last)

// sort the element a[first]..a[last] which reside in the glo.

// array a[1:n] into ascending order; a[n+1] is considered to

// Input: Indexes of the elements to be sorted.

//output: A[first]..A[last] is a sorted rearrangement of the same elements.

{

if (first < last) then.

{

//select a pivot element and partition the array into two subarrays

Pivot = partition(first, last)

// solve the subarrays recursively.

Quicksort (first Pivot-1)

Quicksort (pivot+1, last)

}

else.

{ Print "array cannot be partitioned further"

}

}

```
Algorithm partition(first, last)
// The subarray A[first..last] is partitioned by taking
// first element as pivot.
{
  pivotElement := A[first]
    low := first
    high = last+1
    while(low<high)
     { low++;
       while(A[low] < pivotElement)
       {
         low := low+1
       }
       while(A[high] > pivotElement)
        {
          high = high-1
        }
       if(low<high)
        {
          swap (A[low], A[high])
        }
        }
          A[low] := A[high]
          A[high] := pivot element.
        return high; // return the position of the
                        pivot element.
}
```

Algorithm swap(a, b)

{

temp = a

a = b

b = temp

}.

Example :-

(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)
4	3	7	1	8	12	2	5	$+\infty$
↑								

pivot element = A[1] = 4.

first = 1, last = 8

⇒ if (1 < 8)

pivot = partition (1, 8)

Step 1 :-

pivot element = A[first] = 4

low = 1

high = 9

⇒ 1 < 9, low = 2

A[2] < A[1]

3 < 4

low = 3

A[3] < A[1]

4 < 4

A[8] > pivot

5 > 4

high = 7

A[7] > pivot

2 ≠ 4

4	3	2	1	8	12	7	5
↑ pivot		low				high.	

⇒ 4 3 2 1 8 12 7 5
low high.

⇒ 8 ≠ 4

→ 1 ≠ 4

⇒ 4 3 2 1 8 12 7 5
↑ low

⇒ 1 3 2 (4) 8 12 7 5

At last we get 1 2 3 4 5 7 8 12.

Example 2:-

(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)	(10)
65	70	75	80	85	60	55	50	45	+∞.
↑ p	first=1							last=9	

Partition.

repeat step 1:

=> 65 45 75 80 85 60 55 50 75 45 +∞.
↑ pivot ↑ low=2 ↑ high=9.

2 < 9 so swap.

=>step 2

65 45 75 80 85 60 55 50 70
↑ p ↑ low ↑ low=3 ↑ high=8. ↑ high.

3 < 8, so swap.

step3:-

65 45 50 80 85 60 55 75 70.
P low low high high.

step4:-

65 45 50 55 85 60 80 75 70.
P low low high high.

step5:-

65 45 50 55 60 85 80 75 70.
P low=5 high=6.
high=5 low=6

But 6<5, so don't interchange.

Then swap pivot element and high index value.

pivot = A[high]
A[high] = pivot.

step6:-

60 45 50 55 [65] 85 80 75 70.

Time Complexity of quicksort

(1) **Bestcase** : Quicksort works best if each array is divided into 2 equal subarrays of size $n/2$

→ This generates $\log n$ levels in the recursion tree

The recurrence relation is given by

$$T(n) = \begin{cases} c & n=1 \\ 2T(n/2)+n & n>1 \end{cases}$$

$$T(n) = 2T(n/2)+n.$$

$$= 2\lceil 2T(n/4)+n/2 \rceil + n.$$

$$= 4T(n/4+n+n$$

$$= 4T(n/4)+2n$$

$$= 2^2 T(n/2^2)+2n$$

$$= 2^k T(n/2^k)+kn$$

$$\text{or } \because 2^k = n$$

$$k = \log n.$$

$$\Rightarrow nT(n/n)+n.\log n.$$

$$\Rightarrow nT(1)+n.\log n.$$

$$= c.n + n\log n$$

$$\Rightarrow O(n\log n)$$

$$T(n) = T(n/2 + T(n/2)$$

$$= 2T(n/2)$$

$$= 2\lceil T(n/4 + T(n/4) \rceil$$

$$= 2\lceil 2T(n/4) \rceil$$

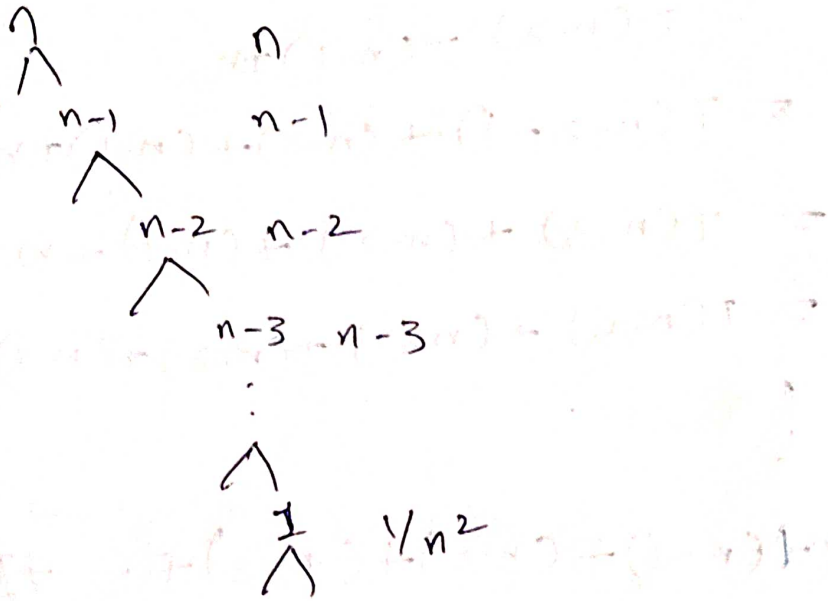
$$= 4T(n/4)$$

$$\vdots$$

$$2^k T(n/4)$$

② Worst case :-

In worst case, the chosen pivot is either the smallest or the largest element in an array. In this case, one part is empty and the other part containing the remaining elements this generats $n-1$ levels in the recursion tree.



→ when quick sort is first called partition takes time $n-1$

→ Quick sort is then called twice, once with an empty array and again with an array of size $n-1$

→ The next call the partition takes $n-2$ times and the next call takes $n-3$ time and the process continues.

=> Recurrence relation for the worst case is

$$T(n) = \begin{cases} C & n=1 \\ T(n-1)+n & n>1 \end{cases}$$

$$T(n) = T(n-1)+n$$
$$= T(n-1-1)+n-1+n$$
$$= T(n-2)+(n-1)+n$$
$$= T(n-2-1)+(n-2)+(n-1)+n$$
$$= T(n-3)+(n-2)+(n-1)+n$$
$$= T(n-4)+(n-3)+T(n-2)+(n-1)+n$$
$$\vdots$$

$$\Rightarrow n+(n-1)+(n-2)+(n-3)+\cdots+1$$

$$\Rightarrow \frac{n(n+1)}{2}$$

$$\Rightarrow \frac{n^2+n}{2}$$

$$\Rightarrow \underline{\underline{O(n^2)}}$$

Average case Analysis :-

In The Average case, the array is partitioned by choosing any random number. In this case at each level some of the partitions are well balanced

$\Rightarrow n+1 \rightarrow$ no. of comparisions.

$\Rightarrow C_n \rightarrow$ Average number of comparisions.

$$C_n(n) = (n+1) + \frac{1}{n}\left[\begin{array}{l} (C_n(0) + C_n(n-1) \\ C_n(1) + C_n(n-2) \\ \vdots \\ C_n(n-1) + C_n(0) \end{array}\right]$$

$$C_n(n) = (n+1) + \frac{2}{n}\left[C_n(0) + C_n(1) + \cdots + C_n(n-1)\right]$$

multiply both sides by n.

$$n C_n(n) = n(n+1) + n \cdot \frac{2}{n}\left[C_n(0) + C_n(1) + \cdots C_n(n-1)\right] \rightarrow (1)$$

Replacing n by n-1 in eq (1)

$$\Rightarrow (n-1) C_n(n-1) = (n-1)(n-1+1) + 2\left[C_n(0) + C_n(1) + \cdots C_n(n-1-1)\right]$$

$$\Rightarrow (n-1) C_n(n-1) = (n-1)(n) + 2\left[C_n(0) + C_n(1) + \cdots + C_n(n-2)\right] \rightarrow (2)$$

Subtracting eq (1) − (2)

$$n C_n(n) - (n-1) C_n(n-1) = n^2 + n - n^2 + n + 2\left(C_n(0) + C_n(1) \cdots + C_n(n-1)\right] - 2\left[C_n(0) + C_n(1) + \cdots + C_n(n-2)\right]$$

$$n C_n(n) - (n-1) C_n(n-1) = 2 C_n(n-1) + 2n$$

$$n C_n(n) = 2 C_n(n-1) + 2n + (n-1) C_n(n-1)$$

41

$\Rightarrow n C_n(n) = C_n(n-1)[2+n-1] + 2n$

$\Rightarrow n C_n(n) = (n+1) C_n(n-1) + 2n$

Divide both sides by $n(n+1)$

$\Rightarrow \frac{n C_n(n)}{n(n+1)} = \frac{C_n(n-1).(n+1)}{n.(n+1)} + \frac{2n}{n(n+1)}$

$\Rightarrow \frac{C_n(n)}{n+1} = \frac{C_n(n-1)}{n} + \frac{2}{n+1}$

Applying the recurrence for $(n-1)$

$\Rightarrow \frac{C_n(n)}{n+1} = \frac{C_n(n-2)}{n-1} + \frac{2}{n-1+1} + \frac{2}{n+1}$

$= \frac{C_n(n-2)}{n-1} + \frac{2}{n} + \frac{2}{n+1}$

$= \frac{C_n(n-3)}{n-2} + \frac{2}{n-1} + \frac{2}{n} + \frac{2}{n+1}$

substitute $n=4$

$\vdots$

$= \frac{C_n(1)}{2} + \frac{2}{3} + \frac{2}{4} + \cdots + \frac{2}{n} + \frac{n}{n+1}$

$= \frac{C_n(1)}{2} + 2\left[\frac{1}{3} + \frac{1}{4} + \cdots + \frac{1}{n} + \frac{1}{n+1}\right]$

$= \frac{C_n(1)}{2} + 2 \sum_{3 \le x \le n+1} \left|\frac{1}{x}\right|$

$\therefore \sum_{3 \le x \le n+1} \frac{1}{x} \le 2 \int_{2 \le x \le n+1} \frac{1}{x}$

Neglect $\frac{c_A(1)}{2}$

$$\frac{c_A(n)}{n+1} \leq 2\int_2^{n+1} \frac{1}{x}$$

$$\leq 2[\log n]_2^{n+1}$$

$$\leq 2[\log n+1 - \log 2]$$

$$\frac{c_A(n)}{n+1} \leq 2\left[\log\left(\frac{n+1}{2}\right)\right].$$

$$c_A(n) \leq 2(n+1)\left[\log\left(\frac{n+1}{2}\right)\right].$$

$f(n) \leq c \cdot g(n)$

$$c_A(n) \leq (n+1)\log(n+1)$$

$$= O(n \cdot \log n)$$

STRASSEN'S MATRIX multiplication.

→ Let A and B be two nxn matrices. The product matrix C = AB is also an nxn matrix where as i, j th element is ~~found~~ formed by taking the element The ith row of A and the jth coloumn of B and multiplying them to get

$$C(i,j) = \sum_{1 \le k \le n} A(i,k)\, B(k,j)$$

~~Algorithm~~ MATMUL(A, B, C, n)

{

~~for~~ i = 1 to n do.

~~for~~ j = 1 to n do.

C(i,j) = 0

~~for~~ k = 1 to n do.

C(i,j) = C[i,j] + A[i,k] * B[k,j];

}

=) matrix multiplication time complexity is $O(n^3)$

→ The divide-and-conquer strategy suggest another way to compute the product of two nxn matrix

=> ~~Assume~~ Assume that n is a power of 2

$n = 2^k$ i.e n = 2, 4, 8, 16, 32 ...

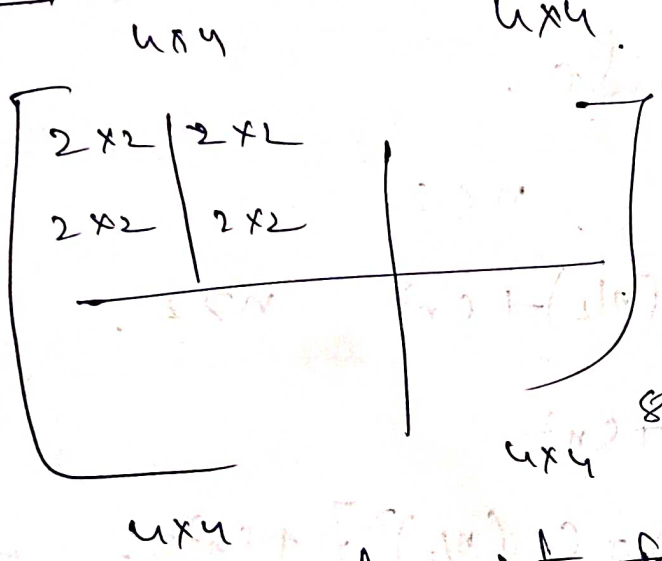
=> Imagine that A and B are each partitioned into four square submatrices each submatrix having dimensions $n/2 \times n/2$

Then the product AB can be computed by using above formula.

~~for the Product of 2x2 matrix.~~

=> divide nxn matrix into submatrices until we get each part as n=2

example:-



→ we divide 8x8 matrix into four parts. later we divide each part into four parts again. we will continue this process until we get all matrices as 2x2 matrices.

→ if AB is

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

$C_{11} = A_{11}B_{11} + A_{12}B_{21}$

$C_{12} = A_{11}B_{12} + A_{12}B_{22}$

$C_{21} = A_{21}B_{11} + A_{22}B_{21}$

$C_{22} = A_{21}B_{12} + A_{22}B_{22}$

To multiply two matrices of order 2 x 2, we requi
8 multiplications and 4 additions

Since two $n/2 \times n/2$ matrices can be added in time cn^2 for some constant c

using DA and C.

$$T(n) = \begin{cases} b & n \leq 2 \\ 8T(n/2) + cn^2 & n > 2. \end{cases}$$

$T(n) = 8T(n/2) + cn^2$

$= 8[8T(n/4) + c \cdot (n/2)^2] + cn^2$ $\quad 2 \quad cn^2$

$= 64T(n/4) + 2cn^2 + cn^2$

$= 64\left[8T(n/8) + c\frac{n^2}{16}\right] + 3cn^2$

$= 512\, T(n/8) + 7cn^2$

$\vdots$

$= 8^k T(n/2^k) + (2^k - 1)c \cdot n^2$

$= n^3 T(n/n) + (n-1) \cdot cn^2$

$= n^3 T(1) + (n-1) \cdot cn^2$

$= n^3 T(1) + cn^3 - cn^2$

$= O(n^3)$

→ matrix multiplication is more expensive than matrix addition.

→ To reduce the time complexity, we use Strassen's matrix multiplication:

→ VOLKER STRASSEN's has discovered a way to compute C_{ij}, using 7 multiplications and 18 additions or subtractions.

→ These methods involve first computing the 7, $n/2 \times n/2$ matrices P, Q, R, S, T, U and V can be computed using 7 multiplication and 10 matrix additions or subtraction.

$$P = (A_{11} + A_{22})(B_{11} + B_{22})$$

$$Q = (A_{21} + A_{22})B_{11}$$

$$R = A_{11}(B_{12} - B_{22})$$

$$S = A_{22}(B_{21} - B_{11})$$

$$T = (A_{11} + A_{12})B_{22}$$

$$U = (A_{21} - A_{11})(B_{11} + B_{12})$$

$$V = (A_{12} - A_{22})(B_{21} + B_{22})$$

$$C_{11} = P + S - T + V$$

$$C_{12} = R + T$$

$$C_{21} = Q + S$$

$$C_{22} = P + R - Q + U.$$

Example:

$$\begin{matrix} A_{11} & A_{12} \\ \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \\ A_{21} & A_{22} \end{matrix} \quad \begin{matrix} B_{11} & B_{12} \\ \begin{bmatrix} 0 & 0 \\ 1 & 1 \end{bmatrix} \\ B_{21} & B_{22} \end{matrix} = \begin{bmatrix} 1\times0+1\times1 & 1\times0+1\times1 \\ 1\times0+1\times1 & 1\times0+1\times1 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

$P = (1+1)(0+1) = 2$

$Q = (1+1)(0) = 0$

$R = (1)(0-1) = -1$

$S = (1)(1-0) = 1$

$T = (1+1)1 = 2$

$U = (1-1)(0+0) = 0$

$V = (1-1)(1+1) = 0$

$\Rightarrow C_{11} = 2+1-2+0 = 1$

$C_{12} = -1+2 = 1$

$C_{21} = 0+1 = 1$

$C_{22} = 2-1-0+0 = 1$

Thus $\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$

The resulting recurrence relation for

$$T(n) = \begin{cases} b & n \le 2 \\ 7T(n/2) + cn^2 & n > 2 \end{cases}$$

$$T(n) = 7T(n/2) + cn^2$$

$$T(n) = 7T(n/2) + cn^2$$

$$= 7\left[7T(n/4) + c\cdot\left[n/2\right]^2\right] + cn^2$$

$$= 49T(n/4) + 7\frac{cn^2}{4} + cn^2$$

$$= 7^2\left[7T(n/8) + c\cdot(n/4)^2\right] + 7\cdot\frac{cn^2}{4} + cn^2$$

$$= 7^3T(n/8) + 7^2\cdot c\cdot\frac{n^2}{16} + 7\cdot\frac{cn^2}{4} + cn^2$$

$$= 7^3T(n/2^3) + cn^2\left(\frac{7}{4}\right)^2 + cn^2\left(\frac{7}{4}\right) + cn^2$$

$$\vdots$$

$$= 7^kT(n/2^k) + cn^2\left[\left[\frac{7}{4}\right]^{k-1} + \left[\frac{7}{4}\right]^{k-2} + \cdots + \frac{7}{4} + 1\right]$$

$$= 7^kT\left(n/2^k\right) + cn^2\left[1 + \frac{7}{4} + \left(\frac{7}{4}\right)^2 + \cdots + \left[\frac{7}{4}\right]^{k-2} + \left[\frac{7}{4}\right]^{k-1}\right]$$

G.S : $\dfrac{a(1-r^k)}{1-r} = \dfrac{1\cdot\left(1-\left(\frac{7}{4}\right)^k\right.}{1-\left(\frac{7}{4}\right)}$

$$= \frac{1-\frac{7^k}{4^k}}{1-7/4} = \frac{7^k}{4^k}.$$

$$= 7^kT(n/2^k) + cn^2\left[\frac{7^k}{4^k}\right]$$

$n = 2^k$.

$k = \log_2 n$.

$$= 7^{\log_2 n}T(n/n) + cn^2\cdot\frac{7^{\log_2 n}}{4^{\log_2 n}}$$

$$= b\cdot n^{\log_2 7} + cn^2\cdot\frac{n^{\log_2 7}}{n^{\log_2 4}}$$

$\log_2 4 = 2$

$$= b\cdot n^{\log_2 7} + cn^2\cdot\frac{n^{\log_2 7}}{n^2}$$

$$= b \cdot n^{\log_2 7} + c \cdot n^{\log_2 7}$$

$$= (b+c) \cdot n^{\log_2 7}$$

$$= n^{\log_2 7}$$

$$= n^{2.81} \Rightarrow \boxed{O(n^{2.81})}$$

Binary search:-

- Let A[1:n] be an array of elements that are in non decreasing order. Consider the problem of determining whether a given element x is present in the array.
- If x is present, we are to determine its position. If x is not in the array, the value 0 is returned.

Let P=(n, Ai -- A1, x) denote an arbitrary instance of the problem, where n is number of elements in the array.

- If x is present, we are to determine its position. If x is not in the array, the value 0 is returned.

Let P=(n, Ai -- A1, x) denote an arbitrary instance of the problem, where n is no. of elements in the array and Ai -- A1 is the array of elements and x is the element to be searched.

→ Divide-and-conquer strategy is used to solve this problem. If the problem(P) size is small i.e if the array consisting of one element, it will take one unit of time to determine the element x is present or not.

If the problem size is not small it can be divided into two equal subarrays. Pick an index in the range (i,j) i.e $q = \frac{(i+j)}{2}$ and compare the element x with A[q].

There are three possibilities.

1.) If (A[q] = x) then the problem is solved and return q.

2.) If (x < A[q]) then the element has to be searched in the subarray A[i:q-1]. Therefore the problem reduces its size.

3.) If (x > A[q]), then x has to be searched for only in the subarray A[q+1:j] and the process is repeated until the element is found.

```
Algorithm Bin-search (A, i, L, x)
// Given an array A[1:n] of elements in non decreasing
// order. x is the element to be searched.
{ if (L = i) then. // if there is one element
  { if (x = A[i]) then return i;
    else return 0;
  }
```

else
{ //Divide the problem into small problems.

mid = $\lceil \frac{(i+1)}{2} \rceil$

if (x = A[mid]) then return mid;
else if (x < A[mid] then
return Binsearch(A, 1, mid-1, x);
else return Bin-search (A, mid+1, i, x);

}

Algorithm IBin-search (A, n, x)
{
low = 1;
high = n;
while (low ≤ high) do
{
mid = $\lceil \frac{(low+high)}{2} \rceil$;
if (x < A[mid]) then
high = mid-1
else
if (x > A[mid]) then
low = mid+1
else return mid;
}
return 0;
}

Iterative Binary Search.

Example:- Let the array consists of elements.

1, 2, 5, 6, 7, 10, 24, 56, 84, 100, 115, 120, 131, 150.

(i) Let x = 150

Here n = 14

Low	high	mid	A[mid]	x ? A[mid]
1	14	7	24	150 > 24
8	14	11	115	150 > 115
12	14	13	131	150 > 131
14	14	14	150	found !!

(ii) Let x = 9

low	high	mid	A[mid]	x ? A[mid]
1	14	7	24	9 < 24
1	6	3	5	9 > 5
4	6	5	7	9 > 7
6	6	6	10	10 > 9
7	6	-	-	-

Since low > high, return Item not found.

Time complexity

The time complexity for binary search technique is generally based on the height of the Binary tree. If the no. of elements are n which is greater than 2^i then the time computation is maximum and is a function of $\log n$.

→ In successful searches, the computing time for binary search in best, average and worst cases are $O(1)$, $O(\log n)$ and $O(\log n)$ respectively.

→ In unsuccessful searches the best, average and worst computing time is $O(\log n)$. Here we are going to find worst case computing time.

→ Every time we will make one comparision and divide given numbers n into n/2

The recurrence relation is

$$T(n) = T(n/2) + 1$$
$$= [T(n/4) + 1] + 1$$
$$= [T(n/8) + 1] + 2$$
$$= T(n/2^3) + 3$$
$$\vdots$$

$= T(n/2^k) + k$

$= T(1) + \log_2 n$ (Let $2^k = n \Rightarrow k = \log_2 n$)

$= 1 + \log_2 n$

$\therefore T(n) = O(\log_2 n)$